

Tips and Tricks for Using FiftyOne in Google Colab

Author: Antonio Rueda-Toicen

Introduction

Over the last year, I have used Google Colab to run computer vision workshops using FiftyOne. The combination of Colab's accessible infrastructure and FiftyOne's powerful dataset visualization capabilities creates an ideal environment for teaching and learning computer vision concepts. However, working with FiftyOne in Colab requires specific tricks to handle the ephemeral nature of cloud instances, manage data persistence, share data quickly, avoid rate limits, and optimize performance.

This guide presents practical solutions I have developed through workshop experience. Each tip addresses real challenges you will encounter when using FiftyOne in collaborative notebook environments.

Of course, you can follow along through the corresponding [Google Colab notebook](#).

Tip: Clean and fast FiftyOne installation with `uv` and `%%capture`

Using `uv` for package installation and `%%capture` to suppress output are great practices for creating clean and efficient Colab notebooks.

In Google Colab, the virtual machine instance you are using is temporary. When you close your browser tab or the notebook becomes idle for too long, the instance is recycled and all installed packages are lost. This is why having a quick way to re-install libraries like FiftyOne is very valuable, and `uv` helps significantly with this.

Why use `uv`?

`uv` is a modern, fast Python package installer and resolver. It is designed to be significantly quicker than traditional tools like `conda`. Using `uv` can drastically reduce the time it takes to set up your environment, especially when dealing with many dependencies. This is useful on Colab notebooks, as they need to be re-configured each time that we close the browser window that runs them.

Why use `%%capture`?

The `%%capture` magic command is used to suppress the standard output and standard error streams of a cell. When installing packages, the output can often be very verbose. Using `%%capture` keeps your notebook clean and focused on the results of your code, rather than the installation process details.

Explicitly specifying package versions

It's a good practice to make the version of the library you are using explicit in your notebook. This is crucial for reproducibility. Different versions of libraries like FiftyOne can have API changes or different dependencies. By specifying the version, you ensure that anyone running your notebook in the future will use the exact same environment you did, saving time and effort in reproducing your work.

```
# We use %%capture to avoid polluting the notebook with the install trace
%%capture
!uv pip install fiftyone==1.8.0

import fiftyone as fo

print(f"FiftyOne version installed: {fo.__version__}")
```

Tip: use Google Drive for persistent storage of data and models across sessions

This is an important workflow for any serious work in Colab. It ensures that your data, models, and datasets are not lost when the virtual machine instance is recycled.

Colab instances are recycled after a period of inactivity (usually around 90 minutes) or when you close your browser tab. Instances also have a maximum lifetime (currently 12 hours), after which they are also recycled. This means any data and work not saved to persistent storage will be lost.

We will use our Google Drive account as persistent storage so that we can save our work. It's important to know that if we share the notebook with collaborators or the outside world, they will only be able to access the data in our Google Drive to which we have granted them open access.

```
# You will be asked to authorize access to Drive after running this.
# Note that this connects to your own Google Drive account.
# It doesn't connect to folders from others unless they have been shared with you.
from google.colab import drive
drive.mount('/gdrive')
```

Permit this notebook to access your Google Drive files?

This notebook is requesting access to your Google Drive files. Granting access to Google Drive will permit code executed in the notebook to modify files in your Google Drive. Make sure to review notebook code prior to allowing this access.

No thanks

[Connect to Google Drive](#)

```
import os
from pathlib import Path
# Notice that this is a Google Drive path
save_path = Path('/gdrive/MyDrive/fiftyone_dataset_curation')
os.makedirs(save_path, exist_ok=True)
```

We specify the location of our MongoDB database for FiftyOne in the folder that we have chosen:

```
# path to the MongoDB database
database_path = save_path / "mongodb"
os.makedirs(database_path, exist_ok=True)
fo.config.database_dir = str(database_path)
```

Let's demo this with the COCO-2017 dataset:

```
import fiftyone.zoo as foz

dataset = foz.load_zoo_dataset(
    "coco-2017",
    split="validation",
    max_samples=200,
)
```

Export the dataset to a Google Drive path:

```
# Export the dataset to a folder
export_dir = save_path / "coco-2017"
dataset.export(
    export_dir=str(export_dir),
    dataset_type=fo.types.FiftyOneDataset,
    # set to True to export the original images alongside annotations
    export_media=True,
)
```

We can later reload the data using the same path:

```
reloaded_dataset = fo.Dataset.from_dir(
    dataset_dir=export_dir,
    dataset_type=fo.types.FiftyOneDataset,
)
```

Tip: use `gdown` to download shared zipped folders

When a dataset is shared as a Google Drive folder, using a tool like `gdown` is more convenient than manually downloading and re-uploading the data to your Colab environment. `gdown` is a command-line utility that allows you to download files and folders directly from a public Google Drive link.

This approach is useful because it automates the download process and can be included directly in your notebook, making your workflow fully reproducible.

First, let's install `gdown` using `uv`, keeping the notebook clean with `%%capture`:

```
# We use %%capture to avoid polluting the notebook with the install trace
%%capture
!uv pip install gdown==5.2.0
```

Next, we'll define the URL of the shared folder and specify a local path where we want to save the data. We will download the same COCO-2017 sample dataset from the link provided earlier.

```
# This assumes 'save_path' is defined from a previous cell.
# Let's ensure it exists.
save_path = Path('/gdrive/MyDrive/fiftyone_dataset_curation')
download_output_path = save_path / "gdown_downloads"
os.makedirs(download_output_path, exist_ok=True)

# The public Google Drive folder URL for the COCO-2017 samples
folder_url =
"https://drive.google.com/drive/folders/1G6JKGm0sy5d5ViEpDktXMxIqq5c12Nrc?usp=drive_link"

# Use gdown to download the entire folder.
# The --folder flag specifies that we are downloading a folder.
# The -O flag sets the output directory.
!gdown --folder "{folder_url}" -O "{download_output_path}"
```

Notice that we have hit a limit:

```
The gdrive folder with url: https://drive.google.com/drive/folders/1fe
    UZBqLJm_20oxLKTxV53SGPZvUYPa49?hl=en has more than 50 files, gdrive
    can't download more than this limit.
```

The file has to be zipped first to bypass this and we must use the shared link to this version.

```

import os
import zipfile

# Define the path to the folder you want to zip
folder_to_zip = '/gdrive/MyDrive/fiftyone_dataset_curation/coco-2017'

# Define the name and path for the output zip file
output_zip_file = '/gdrive/MyDrive/fiftyone_dataset_curation/coco-2017.zip'

# Create a ZipFile object in write mode
with zipfile.ZipFile(output_zip_file, 'w', zipfile.ZIP_DEFLATED) as zipf:
    # Walk through all the files and subdirectories in the folder
    for root, dirs, files in os.walk(folder_to_zip):
        for file in files:
            # Create the full path to the file
            file_path = os.path.join(root, file)
            # Add the file to the zip archive, preserving the directory
            structure = os.path.relpath(file_path, folder_to_zip)
            zipf.write(file_path, structure)

print(f"Folder '{folder_to_zip}' successfully zipped to '{output_zip_file}'")

```

Then we can proceed, assume that we are working now with the zip file [here](#)

```

# --- 1. Setup Paths ---
save_path = Path('/gdrive/MyDrive/fiftyone_dataset_curation')
download_dir = save_path / "gdown_downloads"
os.makedirs(download_dir, exist_ok=True)

# Define the full path for the downloaded zip file
zip_file_path = download_dir / "coco_samples_dataset.zip"

# Define the directory where the contents will be unzipped
unzip_dir = download_dir / "coco_samples_unzipped"
os.makedirs(unzip_dir, exist_ok=True)

# --- 2. Download the ZIP file from Google Drive ---
file_url =
"https://drive.google.com/file/d/1UyL0clgoIPSYDWH1PjLP0IPZoFRQQJDE/view?usp=d
rive_link"

print(f"Downloading dataset from Google Drive to '{zip_file_path}'...")
gdown.download(url=file_url, output=str(zip_file_path), quiet=False)
print("Download complete.")

# --- 3. Unzip the File using the `zipfile` library ---
print(f"Unzipping '{zip_file_path}' to '{unzip_dir}'...")

try:

```

```

        with zipfile.ZipFile(zip_file_path, 'r') as zip_ref:
            zip_ref.extractall(unzip_dir)
            print("Unzipping successful.")
except zipfile.BadZipFile:
    print("Error: The downloaded file is not a valid zip file or is
corrupted.")
except Exception as e:
    print(f"An error occurred during unzipping: {e}")

# --- 4. Load the dataset into FiftyOne ---
dataset_dir = unzip_dir

print(f"Loading FiftyOne dataset from '{dataset_dir}'...")

if (Path(dataset_dir) / "metadata.json").exists():
    dataset = fo.Dataset.from_dir(
        dataset_dir=str(dataset_dir),
        dataset_type=fo.types.FiftyOneDataset,
        name="coco-samples-from-drive"
    )

    print("\nDataset loaded successfully!")
    print(dataset)
else:
    print(f"Error: Could not find a valid FiftyOne dataset at
'{dataset_dir}'.")
    print("Please check the contents of the unzipped folder.")

```

But even then you might get a message like this:

Cannot retrieve the folder information from the link. You may need to change the permission to 'Anyone with the link', or have had many accesses. Check FAQ in <https://github.com/wkentaro/gdown?tab=readme-ov-file#faq>.

“...or have had many accesses” what to do then?

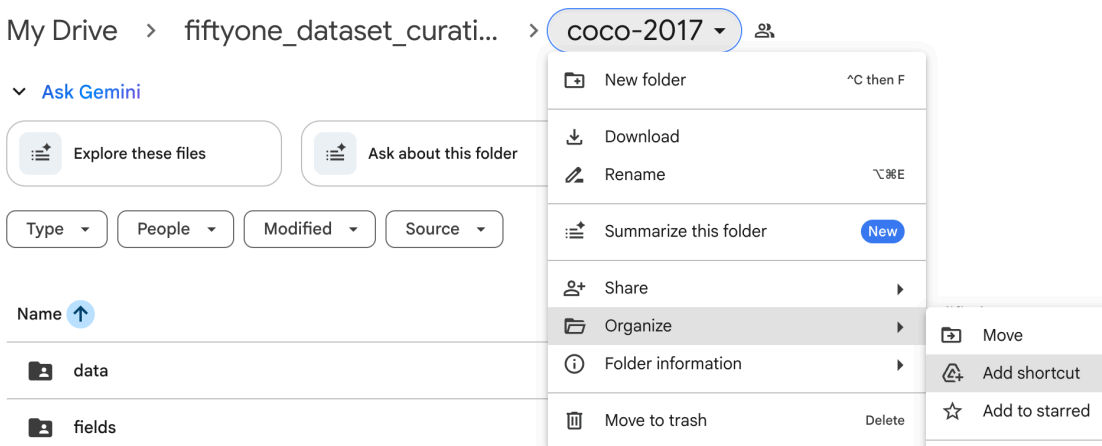
Tip: share datasets instantly by using “Add Shortcut to Drive...”

Adding a shortcut from a shared Google Drive folder to your own account allows you fast access to the data **without having to download it, or compress and decompress it**. It's a much more immediate option than using compressed folders with gdown and importantly, **doesn't run into rate limits**.

Follow these steps to add a shortcut to a shared Google Drive folder in your own Drive for quick access.

You can try the following steps using this public access link to samples from the COCO-2017 validation set that I created, [link here](#).

1. Open Google Drive
 - Go to drive.google.com and sign in with your Google account if you are not already signed in.
2. (Optional) Locate the Shared Folder
 - If someone shared the folder with you through an invitation (instead of a link), look under the **“Shared with me”** section on the left sidebar.
3. Add the Shortcut
 - Right-click (or two-finger click on a touchpad) on the folder name in Google Drive.



- Select **Organize > Add shortcut to Drive** from the menu.
4. Choose Shortcut Location
 - A dialog box will appear. Select **My Drive** or any other folder in your Drive where you want the shortcut to appear.
 - Click **Add shortcut** to confirm.
 5. Access Your Shortcut
 - Go to the location you selected (e.g., “My Drive”).
 - You will see the shortcut there, marked with a small arrow icon.
 - You will be able to refer to this folder using the path specified:

```
from pathlib import Path
import os
path = Path("/gdrive/MyDrive/<folder_name>")
# This should print the filenames inside the folder
os.listdir(path)
```

Shortcuts in Google Drive are just pointers to the original folder: they do not make a copy or use your storage quota. Any changes you make to files within the shortcut are reflected in the original folder. They are an instant way to share data and come in handy when working with collaborative datasets. Whenever I run a workshop and need to have 20+ people to access a dataset simultaneously, I rely on the shortcuts for they can download the data without hitting rate limits, a common issue when working with gdown or datasets hosted on HuggingFace Hub.

Tip: Save your FiftyOne setup on Drive by modifying the `fo.config` file

We can give full persistence to our Google Colab environment. We can use the Google Drive path for our downloads, models, and MongoDB database by modifying `fo.config`:

```
# Check the state of fo.config before doing any modification
print(fo.config)

# Where we will download the data when using the FiftyOne dataset zoo
# https://docs.voxel51.com/dataset_zoo/index.html
dataset_zoo_path = save_path / "fo_dataset_zoo"
os.makedirs(dataset_zoo_path, exist_ok=True)
fo.config.dataset_zoo_dir = str(dataset_zoo_path)

# path to the MongoDB database
database_path = save_path / "mongodb"
os.makedirs(database_path, exist_ok=True)
fo.config.database_dir = str(database_path)

models_path = str(save_path / "models")
os.makedirs(models_path, exist_ok=True)
fo.config.model_zoo_dir = models_path

model = foz.load_zoo_model("clip-vit-base32-torch")
```

This allows us to keep our work when the Google Colab instance gets recycled.

Tip: add your HuggingFace token as a secret in Colab

Adding a HuggingFace token as a secret to your Google Colab notebook allows us to share our work using models and data from HF without exposing our personal information.

Why You Need a Token

- **Access Restricted Models:** Many powerful models on HuggingFace, like Meta's DINOv3 family, are "gated," meaning we must agree to their terms before we can download them. Our token verifies that we have the necessary permissions.

- **Avoid Rate Limiting:** Without a token, we are an anonymous user and subject to strict download limits. Using a token identifies us and grants us higher download rates, preventing interruptions in our workflow.
- **Upload Your Own Work:** If we plan to share our fine-tuned models or datasets with the community, our token acts as our credential, allowing us to upload and manage content on the HuggingFace Hub.

Step 1: Create a HuggingFace Account

1. Navigate to the [HuggingFace website](#).
2. Click the “**Sign Up**” button located in the top-right corner.
3. You can register using your email or by linking your Google or GitHub account.
4. Complete the process by verifying your email address.

Step 2: Generate Your Access Token

1. Once logged in, click on your profile picture at the top-right and select “**Settings**” from the dropdown menu.
2. In the left sidebar, click on “**Access Tokens.**”
3. Click the “**New token**” button.
4. Give your token a descriptive name (e.g., “Google Colab Projects”).
5. Assign it the “**Write**” role to enable both downloading and uploading.
6. Click “**Generate a token.**”
7. Copy the generated token and store it in a secure location, as you will not be able to see it again.

Step 3: Use Your Token in Google Colab

The most secure way to use your token in Google Colab is with the built-in Secrets manager. This prevents your token from being exposed in the code.

1. **Install Required Libraries:** In a Colab cell, run the following command to install the necessary HuggingFace libraries:

```
!pip install -q huggingface_hub transformers
```

2. **Store the Token in Colab Secrets:**

- In your Colab notebook, click the **key icon** () in the left sidebar.
- Click “**Add a new secret.**”
- For the **Name**, enter HUGGINGFACE_TOKEN.
- In the **Value** field, paste your copied access token.
- Ensure the “**Notebook access**” toggle is enabled.

3. **Access the Token in Your Notebook:** Use the following code to load your token from Colab Secrets into your environment.

```

import os
from google.colab import userdata

# Load the token from Colab secrets
os.environ["HUGGINGFACE_TOKEN"] = userdata.get('HUGGINGFACE_TOKEN')

# You can now use the HuggingFace API
from huggingface_hub import HfApi
api = HfApi()

# Example: List your models to verify the token is working
# Replace "your-username" with your actual HuggingFace username
my_models = api.list_models(author="your-username")
print("Connected and fetched models.")

```

Security Best Practices

- **Never expose your token** in plain text in your notebook cells or share it publicly.
- **Use the Secrets manager** as the primary method for handling tokens in Colab.
- If you believe your token has been compromised, go to your HuggingFace settings and revoke it.

Getting a dataset from HuggingFace's Hub

Having the HF token set up, interaction with the platform is much smoother:

```

from fiftyone.utils.huggingface import load_from_hub

other_datasets_path = save_path / "other_datasets"
os.makedirs(other_datasets_path, exist_ok=True)
fo.config.default_dataset_dir = str(other_datasets_path)

# We are less likely run into rate limiting now
curated_minist_dataset = load_from_hub("Voxel51/curated-mnist",
                                       max_samples=100)

```

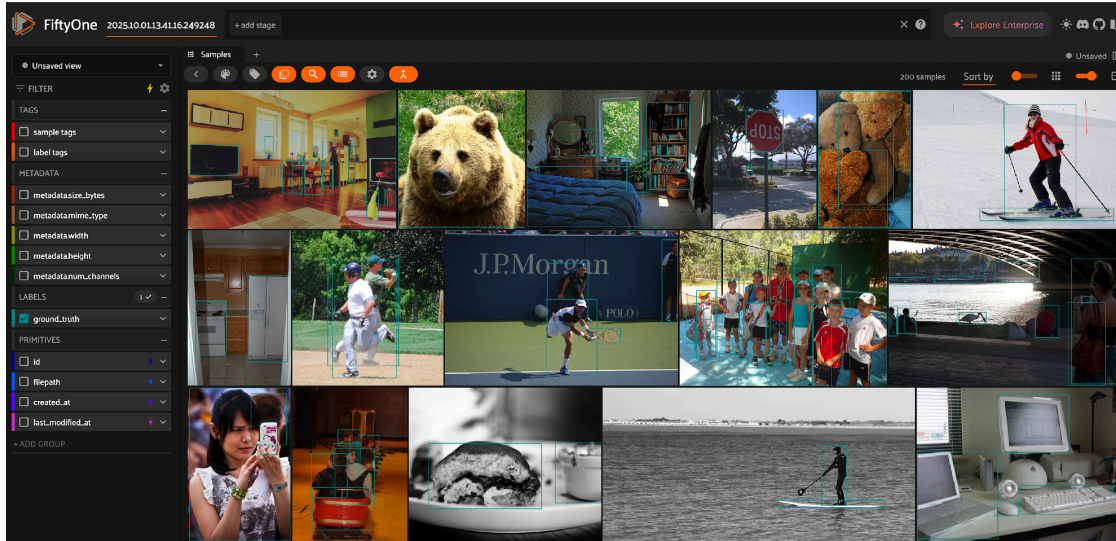
Tip: control the launch of the FiftyOne app in a separate browser tab

Launching the FiftyOne app in a separate browser tab provides a full window view. This is often more convenient and provides a better user experience than launching the app within a notebook cell.

```

# Passing auto=False prevents the app from launch on its own notebook cell
session = fo.launch_app(dataset, auto=False)
# print session.url gives us a nice URL that we can click on
print(f"Just click here to get to the app (whenever you want, no auto launch) {session.url}")

```



`session.open_tab()` is another option, however it fires the app, which can feel disruptive or distracting. I prefer to use `print(session.url)` to control when I access the FiftyOne app.

```
# session.open_tab() is another option, we just don't get to see the URL here
# session.open_tab()
```

Tip: keep a single instance of the app running

Each time you call `fo.launch_app()`, FiftyOne launches a new server process in the background to power the app. While you can launch multiple apps, it's a good practice to manage a single instance to conserve resources and avoid confusion. If you see the FiftyOne app "flickering" it's often because we have two instances of it running at the same time.

When you launch the app, store the returned `session` object. You can use this object to control the app. If you have a session running and want to get a handle on it again, you can call `session.url` again, and FiftyOne will give you a handle to the existing session without creating a new one.

Let's see this in action. First, we launch the app and get our session object:

```
# If it's flickering it's because we have called fo.launch_app() inside the
# notebook
# This will shut down the App server associated with the previous session
session.close()

# Launch the app and store the session.
session = fo.launch_app(dataset, auto=False)
print(session.url)
```

Now you can perform other operations in your notebook. When you want to interact with the app again, for example, to load a new view, you can just use the `session` object:

```
from fiftyone import ViewField as F
```

```
dog_view = dataset.filter_labels("ground_truth", F("label") == "dog")
session.view = dog_view
```

If you close the App's cell or tab, the underlying Python process is still running. You can open a new tab to the App's URL. If you need to stop the App server, you can close the session.

Tip: use the GPU to run inference faster

To speed up inference and other computations in FiftyOne, you can leverage the power of a GPU. Google Colab provides free access to GPUs, which can be enabled by going to:

Runtime -> Change Runtime Type -> T4 GPU

This gives us access to an NVIDIA T4 GPU with about 16 GB of VRAM. Inference will run much faster with it enabled, this affects how we will be able to obtain predictions or embeddings out of our models. We can try this with a speed test:

```
import torch
# Check if the GPU is available
torch.cuda.is_available()
## Prints True if we have the GPU enabled
```

Let's do a benchmark:

```
import time
import fiftyone.brain as fob
```

```
# Helper: measure time for computing embeddings into a field
def time_compute_embeddings_and_project(model, label="gpu"):
    start = time.time()
    # Use Brain to both compute embeddings and reduce dimensionality
    res = fob.compute_visualization(
        dataset,
        model=model,
        embeddings="clip_embeddings_" + label,
        brain_key="vis_" + label,
        method='pca',
        batch_size=16,
    )
    end = time.time()
    return end - start
```

```
# Load CLIP (OpenAI ViT-B/32) from the model zoo
clip_model = foz.load_zoo_model("clip-vit-base32-torch")
print("Model has embeddings:", getattr(clip_model, "has_embeddings", None))
```

```
# Benchmark on GPU (if available)
gpu_time = None
if torch.cuda.is_available():
    # Many FiftyOne zoo models run on GPU when available;
    # timing reflects GPU execution.
    gpu_time = time_compute_embeddings_and_project(clip_model, label="gpu")
    print(f"GPU time (s): {gpu_time:.2f}")

# Force CPU run by moving model to CPU (and/or disabling CUDA)
# Re-load a fresh model instance to avoid device cross-talk
clip_model_cpu = foz.load_zoo_model("clip-vit-base32-torch")
cpu_time = time_compute_embeddings_and_project(clip_model_cpu, label="cpu")
print(f"CPU time (s): {cpu_time:.2f}")
```

Tip: use the extra RAM if you have it

When working with large datasets, with high-resolution images or videos, you may run into memory limitations on the standard Colab runtime (which provides around 12GB of RAM). Colab Pro offers a “High-RAM” runtime that provides more memory (around 25GB or more), which can be essential for memory-intensive tasks.

To enable a high-RAM runtime, go to:

Runtime -> Change Runtime Type -> Runtime shape -> High-RAM

This option will give your notebook access to a machine with a larger memory capacity, allowing you to load and process more data without the session crashing. This is useful when you need to load a large number of samples into FiftyOne, compute embeddings for your entire dataset, or perform other memory-heavy operations.

FiftyOne is snappy and quick even on low resources. You can try loading a bigger version of the COCO-2017 dataset and compare the feel of the app when you have the extra RAM available:

```
import psutil

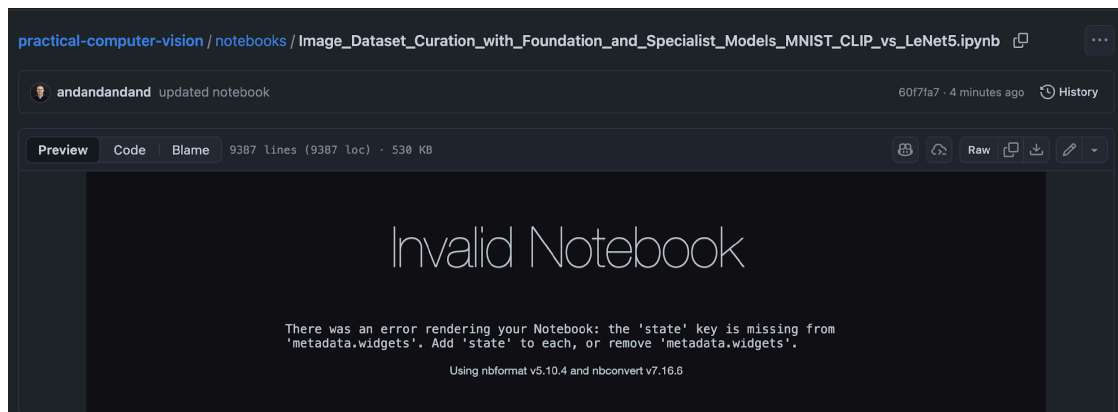
# Check available RAM
total_ram_bytes = psutil.virtual_memory().total
# Convert 16GB to bytes
sixteen_gb_bytes = 16 * 1024 * 1024 * 1024

if total_ram_bytes > sixteen_gb_bytes:
    try:
        big_dataset = foz.load_zoo_dataset(
            "coco-2017",
            split="validation",
            max_samples=5000, # Loading 5000 samples requires more memory
        )
        print(f"Loaded a larger dataset with {len(big_dataset)} samples.")
        # You can now launch the app with this larger dataset
```

```
# session = fo.launch_app(big_dataset)
# print(session.url)
except Exception as e:
    print(f"An error occurred: {e}")
    print("This may be due to insufficient RAM. Try switching to a
High-RAM runtime.")
else:
    print("Skipping loading the larger dataset: Insufficient RAM (less than
16GB) available.")
```

Tip: ignore Invalid Notebook messages on GitHub using the githubtocolab URL trick

Many Jupyter and Colab notebooks produce artifacts that don't allow them to render on GitHub previews.

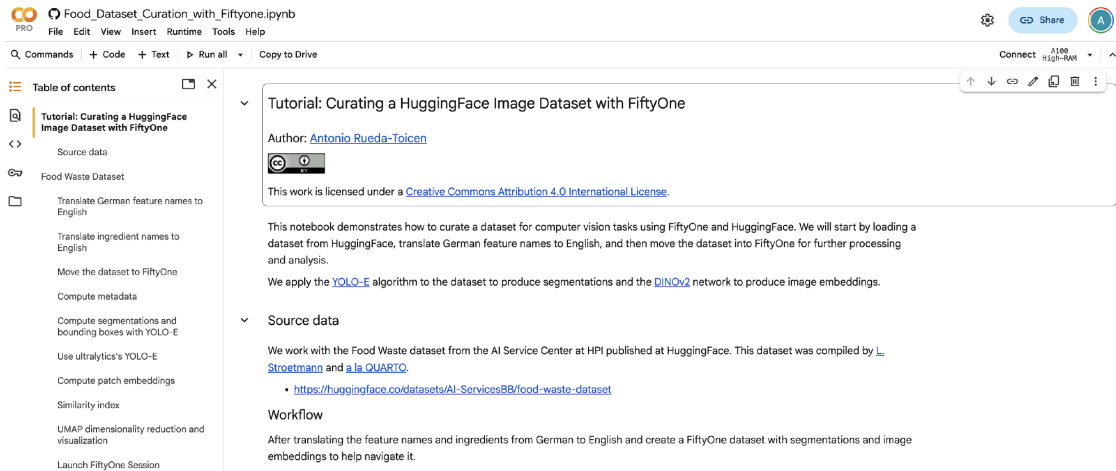


Here we have an example:

https://github.com/andandandand/practical-computer-vision/blob/main/notebooks/Food_Dataset_Curation_with_Fiftyone.ipynb

If we change the URL from github.com to githubtocolab.com, the same renders. Try it out!

https://githubtocolab.com/andandandand/practical-computer-vision/blob/main/notebooks/Food_Dataset_Curation_with_Fiftyone.ipynb



Conclusion

These tips represent practical solutions to common challenges when using FiftyOne in Google Colab environments. The ephemeral nature of cloud instances requires careful attention to data persistence, while the collaborative aspects of notebooks benefit from clean installation practices and proper resource management.

The integration of Google Drive provides a foundation for persistent workflows, while tools like gdown and Google Drive shortcuts enable efficient data sharing. GPU acceleration and memory optimization techniques ensure that FiftyOne's powerful visualization and analysis capabilities can be used at scale.

These tips and tricks have proven valuable for me in many workshops and training sessions. They transform FiftyOne in Colab from a temporary experimentation platform into a robust environment for serious computer vision work. Whether you're teaching, learning, or conducting research, these techniques will help you build reliable and reproducible workflows that leverage the full potential of both platforms.